

UniTorV E-Vote

**Progettazione e Sviluppo di una Piattaforma Sicura per la
Gestione di Riunioni e Votazioni Aziendali su Blockchain**

Candidati:
Christian Sfeir, Mario Chiappini, Francesco Cosciotti

Indice

1	Panoramica del Progetto	4
2	Use Case	5
2.1	Attori del Sistema	5
2.2	Tabella dei Casi d'Uso	5
2.3	Descrizione Dettagliata dei Casi d'Uso Principali	6
2.3.1	UC-01 – Autenticazione al Sistema	6
2.3.2	UC-02 – Creazione di una Riunione Aziendale	6
2.3.3	UC-03 – Gestione delle Sessioni di Voto	7
2.3.4	UC-04 – Partecipazione e Registrazione del Voto	7
3	Sequence Diagram	8
3.1	Sequenza: Autenticazione Utente	8
3.2	Sequenza: Creazione di una Riunione	8
3.3	Sequenza: Recupero dello Storico Riunioni	9
3.4	Sequenza: Registrazione del Voto e Scrutinio	9
4	Requisiti del Sistema	10
4.1	Requisiti Funzionali	10
4.2	Requisiti Non Funzionali	11
4.3	Requisiti Strutturali e Architetturali	12
5	Architettura del Sistema	13
5.1	Architettura Client-Server e Separazione dei Livelli	13
5.2	Gestione dell'Autenticazione e delle Sessioni	14
5.3	Comunicazione tramite API REST	15
6	Componenti Architetturali	16
6.1	Frontend — Web Application	16
6.1.1	Sala di Voto Virtuale (<code>voting_room.js</code>)	16
6.1.2	Pagina dei Risultati (<code>results.js</code>)	16
6.2	Backend Server — Node.js con Express.js	16
6.2.1	Gestione dell'Autenticazione (<code>login_route.js</code>)	16
6.2.2	Gestione delle Riunioni (<code>meetings_route.js</code>)	16
6.2.3	Gestione delle Votazioni (<code>votation_route.js</code>)	17
6.2.4	Gestione dei Verbali PDF (<code>files_route.js</code>)	17
6.2.5	Gestione Utenti (<code>users_route.js</code>)	17
6.2.6	Middleware e Cross-Cutting Concerns	17

7	Sistema di Logging Applicativo	18
7.1	Architettura del Logger	18
7.2	Recupero dei Log	18
8	Mappatura degli Eventi di Log per Route	18
8.1	Autenticazione – login_route.js	19
8.2	Middleware di Autenticazione – auth_middle.js	19
8.3	Middleware Recupero Utente – fetch_user_middle.js	20
8.4	Middleware Verifica Voto – check_vote_middle.js	20
8.5	Gestione Riunioni – meetings_route.js	20
8.6	Gestione File – files_route.js	21
8.7	Gestione Utenti – users_route.js	21
8.8	Verifica Ruolo – role_route.js	21
9	Riepilogo Statistico degli Eventi di Log	22
10	Analisi di Sicurezza del Sistema di Logging	22
10.1	Punti di Forza	22
11	Threat Modeling — Modello STRIDE	23
11.1	Spoofing — Impersonificazione	23
11.2	Tampering — Manipolazione dei Dati	23
11.3	Repudiation — Ripudio delle Azioni	23
11.4	Information Disclosure — Divulgazione di Informazioni	23
11.5	Denial of Service — Interruzione del Servizio	24
11.6	Elevation of Privilege — Escalation dei Privilegi	24
12	Attack Surface Analysis	25
12.1	Autenticazione e Gestione delle Sessioni	26
12.2	Comunicazioni e Canali di Rete	27
13	Mitigation Strategies	28
13.1	Autenticazione Sicura e Protezione delle Credenziali	28
13.2	Protezione delle Comunicazioni	28
13.3	Integrità tramite Blockchain e Controllo degli Accessi	28
13.4	Logging, Auditing e Monitoraggio	28
14	Definizione dell'infrastruttura	30
14.1	Definizione delle Zone (IEC 62443)	30
14.2	Asset Inventory	31
14.3	Superficie Osservabile	32
14.4	Grafo dell'Infrastruttura	33
15	Definizione della Baseline e Analisi delle Deviazioni	34
15.1	Valutazione del Dataset Osservato	35
15.2	Step 3: Dati Grezzi	35
15.3	Step 4: Valutazione Preliminare	36
15.4	Step 5: Classificazione Finale e Dato Utile	37
16	Endpoint rilevati nel log (28/05/2026)	39
17	Baseline path — CEO	40

18 Baseline path — MEMBRO	41
19 Albero decisionale — Autenticazione e routing per ruolo	42
20 Albero decisionale — Ciclo di votazione	43
21 Riepilogo anomalie e raccomandazioni	44
21.1 Errori critici	44
21.2 Osservazioni sul flusso	44
21.3 Legenda colori	44

Panoramica del Progetto

Il presente documento costituisce la documentazione tecnica aggiornata relativa allo sviluppo della piattaforma *UniTorV E-Vote*, un sistema di voto elettronico aziendale progettato con l'obiettivo primario di garantire sicurezza, trasparenza e verificabilità nell'ambito della gestione delle riunioni e delle votazioni interne all'organizzazione.

Il progetto nasce dall'esigenza di superare i limiti intrinseci dei processi di votazione tradizionali — oggi in contesti aziendali di grandi dimensioni gestiti digitalmente con strumenti insufficienti — introducendo un'infrastruttura tecnologica avanzata, scalabile e accessibile tramite browser: un backend applicativo basato su Node.js con Express.js e un layer blockchain dedicato alla registrazione immutabile sia dei voti, sia dei verbali certificati, sia dei profili utente.

La piattaforma è progettata per operare in un contesto aziendale strutturato, in cui coesistono due profili utente distinti: il CEO Amministratore, con privilegi di amministrazione completa del sistema, e il Membro, con accesso limitato alle funzionalità operative di partecipazione al voto. Tale distinzione dei ruoli è implementata sia a livello di interfaccia utente che a livello di logica applicativa lato server, con l'applicazione di meccanismi di autorizzazione basati su token JWT (JSON Web Token) e middleware di controllo degli accessi.

L'integrazione con la tecnologia blockchain rappresenta l'elemento architetture più rilevante dal punto di vista della cybersecurity: ogni voto espresso durante una sessione assembleare viene registrato su blockchain, producendo un record immutabile e crittograficamente verificabile. L'intera persistenza del sistema — voti, riunioni, verbali e profili utente — è affidata alla blockchain, eliminando qualsiasi dipendenza da database relazionali esterni. Questo approccio garantisce le tre proprietà fondamentali di sicurezza dell'informazione — Confidenzialità, Integrità e Disponibilità (CIA Triad) — applicate specificamente al dominio del voto elettronico aziendale.

Gli obiettivi primari della piattaforma possono essere sintetizzati come segue:

- Garantire l'autenticità dell'identità degli utenti tramite un sistema di autenticazione sicuro basato su credenziali cifrate e token di sessione JWT, con recupero dei profili interamente dalla blockchain.
- Assicurare l'integrità e l'immutabilità dei voti e dei verbali registrati mediante tecnologia blockchain, eliminando qualsiasi possibilità di alterazione retroattiva.
- Proteggere la confidenzialità del voto del singolo partecipante attraverso l'anonimizzazione crittografica dell'identità e la ponderazione per quota assembleare.
- Fornire un meccanismo di scrutinio server-side trasparente, il cui risultato viene sigillato sulla blockchain e reso consultabile da tutti i partecipanti.
- Offrire un'interfaccia utente intuitiva e accessibile, adatta all'utilizzo in contesti aziendali formali da parte di utenti con diversi livelli di competenza tecnica.

Use Case

La seguente sezione descrive i principali scenari di utilizzo della piattaforma *UniTorV E-Vote*, identificando gli attori coinvolti, le precondizioni necessarie all'esecuzione di ciascun caso d'uso e gli esiti attesi. I casi d'uso sono stati definiti a partire dall'analisi dei requisiti funzionali e delle interazioni previste tra gli utenti e il sistema.

Attori del Sistema

Il sistema prevede due categorie principali di utenti, ciascuna con un insieme distinto di privilegi e responsabilità:

- **CEO Amministratore:** figura con accesso completo alle funzionalità di gestione del sistema. Il CEO può creare e gestire riunioni, avviare e chiudere sessioni di voto, accedere allo storico completo delle assemblee tramite il pannello di audit, caricare verbali certificati in formato PDF (registrati sulla blockchain), visualizzare l'elenco completo degli utenti registrati e consultare i risultati sigillati di ogni votazione.
- **Membro Aziendale:** figura con accesso limitato alle funzionalità operative. Il membro può partecipare alle sessioni di voto per le riunioni in cui è incluso nella lista dei partecipanti, consultare lo storico delle proprie partecipazioni, visualizzare i risultati delle votazioni concluse e gestire i dati del proprio profilo personale.

Tabella dei Casi d'Uso

Tabella 1: Tabella riassuntiva dei casi d'uso

ID	Attore	Scenario	Precondizioni	Esito
UC-01	CEO / Membro	Autenticazione al sistema tramite credenziali	Utente registrato sulla blockchain	Accesso alla dashboard di ruolo, JWT emesso
UC-02	CEO	Creazione di una nuova riunione aziendale	CEO autenticato	PDF verbale registrato su blockchain, metadati riunione registrati su blockchain
UC-03	CEO	Apertura/chiusura sessione di votazione	Riunione attiva esistente, CEO autenticato	Stato votazione aggiornato su blockchain; a chiusura lo scrutinio viene sigillato
UC-04	Membro	Partecipazione e voto in una sessione attiva	Sessione aperta, membro nella lista partecipanti	Voto ponderato per quota registrato su blockchain con identità anonimizzata SHA-256
Continua nella pagina successiva...				

Tabella 1 – Continua dalla pagina precedente

ID	Attore	Scenario	Precondizioni	Esito
UC-05	CEO / Membro	Visualizzazione risultati di una votazione chiusa	Votazione nello stato closed	Risultati aggregati (favorevoli, contrari, astenuti, somma algebrica quote) visualizzati
UC-06	CEO	Consultazione archivio riunioni (Audit Report)	CEO autenticato	Lista completa riunioni con stato e accesso ai dettagli
UC-07	Membro	Visualizzazione storico partecipazioni personali	Membro autenticato	Lista riunioni filtrata per email del membro
UC-08	CEO	Visualizzazione lista utenti registrati	CEO autenticato	Elenco utenti con nome, email, wallet e ruolo recuperato dalla blockchain
UC-09	CEO / Membro	Download verbale PDF di una riunione	Utente autenticato, riunione con verbale caricato	PDF decodificato dalla blockchain e inviato al browser
UC-10	CEO / Membro	Gestione del profilo e logout sicuro	Utente autenticato	Sessione terminata, localStorage pulito

Descrizione Dettagliata dei Casi d'Uso Principali

2.3.1 UC-01 – Autenticazione al Sistema

L'utente accede alla pagina di login e inserisce le proprie credenziali (indirizzo wallet come identificativo e password come chiave di accesso). Il frontend trasmette i dati in formato JSON al backend tramite una richiesta HTTP POST all'endpoint `/api/v1/loginCheck`. Il backend recupera l'utente dalla blockchain tramite il middleware `fetchUserFromBlockchain`, verifica la corrispondenza della password tramite funzione `bcrypt.compare()`, genera un token JWT in caso di successo e restituisce al client il token e i dati del profilo. Il frontend memorizza il token e i dati utente nel *localStorage* per le successive chiamate autenticate, reindirizzando l'utente alla dashboard appropriata in base al ruolo (CEO o Membro).

Il sistema supporta anche la verifica di un token JWT già presente nell'header **Authorization**: se il token è incluso nella richiesta POST a `/api/v1/loginCheck`, il backend lo verifica direttamente senza richiedere la password, consentendo il rinnovo trasparente della sessione.

2.3.2 UC-02 – Creazione di una Riunione Aziendale

Il CEO, dalla propria dashboard, accede al pannello di creazione riunione. Compila il modulo specificando il titolo dell'assemblea, le date di inizio e fine, la lista dei partecipanti (identificati tramite indirizzi email) e carica il verbale certificato in formato PDF. Il frontend esegue una sequenza di chiamate API: la prima all'endpoint `/api/v1/uploadVerbale`, che riceve il file PDF tramite `multipart/form-data`, lo converte in stringa Base64 e lo salva sulla blockchain nella classe **Verbale** con chiave `meetingKey`; la seconda all'endpoint `/api/v1/create-meeting`, che registra i metadati della riunione (titolo, date, partecipanti, numero di votazioni) nella classe

`Reunion` e inizializza automaticamente lo stato di tutte le votazioni previste (impostando ciascuna a `false`, ovvero non ancora aperta) nella classe `Votation` con chiave composta `[meetingKey, "status"]`.

2.3.3 UC-03 – Gestione delle Sessioni di Voto

Il CEO, dalla pagina di dettaglio di una riunione attiva, può aprire una singola votazione (impostando il suo stato a `true`), consentendo ai membri nella lista partecipanti di accedervi. Al momento della chiusura (impostazione dello stato a `"closed"`), il sistema esegue automaticamente lo scrutinio lato server chiamando l'endpoint `/api/v1/validation-vote`: viene recuperato l'intero storico dei voti registrati sulla blockchain per quella votazione, viene calcolata la somma algebrica delle quote (favorevoli positivi, contrari negativi, astenuti zero), il conteggio per categoria e il risultato finale viene sigillato come nuovo record blockchain, sovrascrivendo il payload corrente della votazione e rendendolo immutabile.

2.3.4 UC-04 – Partecipazione e Registrazione del Voto

Durante una sessione di voto attiva, il sistema notifica i membri autorizzati tramite un popup di convocazione nella dashboard personale. Prima di consentire l'accesso alla sala di voto virtuale, il frontend verifica che la specifica votazione sia nello stato `true` chiamando `/api/v1/get-votations-status`: in caso contrario, l'accesso viene bloccato lato client. Il membro esprime la propria preferenza (favorevole, contrario, astenuto). Il frontend trasmette il voto al backend tramite `/api/v1/add-vote`: il backend verifica che il membro non abbia già votato (middleware `checkIfAlreadyVoted`), recupera dalla blockchain la quota assembleare del votante, calcola il valore ponderato del voto (quota per favorevole, $-$ quota per contrario, 0 per astenuto), anonimizza l'identità del votante tramite hash SHA-256 dell'email e registra il payload sulla blockchain.

Sequence Diagram

I seguenti sequence diagram descrivono in modo formale le principali sequenze di interazione tra i componenti architetturali del sistema: Frontend (Browser), Backend (Express.js) e Blockchain Layer. Ogni diagramma rappresenta un flusso operativo critico della piattaforma, con particolare attenzione ai passaggi che coinvolgono operazioni di sicurezza.

Sequenza: Autenticazione Utente

La sequenza di autenticazione ha inizio quando l'utente sottomette il modulo di login nel browser. Il frontend raccoglie le credenziali (*id_wallet* e password) e invia una richiesta HTTP POST all'endpoint `/api/v1/loginCheck` del backend. Il middleware `fetchUserFromBlockchain` intercetta la richiesta e recupera il profilo utente direttamente dalla blockchain, risolvendo prima la corrispondenza email/wallet tramite la tabella di mapping e poi i dati completi dell'utente tramite il relativo *id_wallet*.

Il backend confronta quindi la password fornita con il valore hashed memorizzato sulla blockchain tramite `bcrypt.compare()`. In caso di corrispondenza, genera un token JWT firmato con la chiave segreta del server e lo include nella risposta HTTP 200 insieme all'oggetto dati dell'utente (nome, cognome, email, wallet address, ruolo/classe). Il frontend persiste questi dati nel `localStorage` e reindirizza l'utente alla dashboard di competenza. In caso di credenziali errate, il server restituisce HTTP 401.

Le successive richieste autenticate includono il token JWT nell'header di autorizzazione (**Bearer Token**). Il backend, tramite il middleware `verifyToken`, verifica la validità e la firma del token prima di concedere l'accesso alle rotte protette, implementando un meccanismo di controllo degli accessi stateless. Il frontend chiama altresì `/api/v1/check-role` all'apertura di pagine sensibili (es. `meeting.html`) per verificare lato server se l'utente possiede privilegi CEO, rendendo il controllo di accesso indipendente dai dati in `localStorage`.

Sequenza: Creazione di una Riunione

La sequenza di creazione di una riunione prevede due fasi distinte, gestite come operazioni asincrone successive con un timeout globale di 15 secondi gestito tramite `AbortController`.

Nella prima fase, il frontend invia il file PDF del verbale al backend tramite una richiesta HTTP POST `multipart/form-data` all'endpoint `/api/v1/uploadVerbale`. Il backend gestisce il caricamento tramite il middleware Multer con storage in memoria (`memoryStorage`), evitando la scrittura su disco: il buffer binario del file viene convertito in stringa Base64 e registrato sulla blockchain nella classe `Verbale` con chiave `meetingKey`. Il PDF non tocca mai il filesystem del server.

Nella seconda fase, il frontend costruisce l'oggetto JSON della riunione e lo trasmette all'endpoint `/api/v1/create-meeting`. Il backend serializza i metadati della riunione e li registra sulla blockchain nella classe `Reunion` con chiave univoca `reunion_[timestamp]`. Contestualmente, inizializza automaticamente le strutture di stato delle votazioni nella classe `Votation`, impostando ogni votazione prevista allo stato `false` (non aperta).

Sequenza: Recupero dello Storico Riunioni

Il recupero dello storico è delegato interamente al backend tramite l'endpoint unificato `GET /api/v1/meetings`. La logica di aggregazione è centralizzata lato server: il backend recupera tutte le chiavi della classe `Reunion` dalla blockchain, filtra quelle con prefisso `reunion_`, recupera il payload JSON di ciascuna e filtra le riunioni in base al ruolo del richiedente (il CEO vede tutte le riunioni; il Membro vede solo quelle in cui la propria email è presente nell'array `partecipanti`). Il risultato è restituito già ordinato per timestamp decrescente.

Sequenza: Registrazione del Voto e Scrutinio

La registrazione del voto rappresenta la sequenza più critica dal punto di vista della sicurezza. Il backend, ricevuta la richiesta all'endpoint `/api/v1/add-vote`, esegue in sequenza i seguenti controlli: verifica dell'autenticità del token JWT tramite `verifyToken`, verifica dell'assenza di un voto precedente tramite il middleware `checkIfAlreadyVoted`, recupero della quota assembleare del votante dalla blockchain tramite `fetchUserFromBlockchain`. Superati i controlli, il backend calcola il valore ponderato del voto, anonimizza l'identità del votante tramite hash SHA-256 dell'email e registra il payload `{partecipante : hash, value : valoreVoto}` sulla blockchain tramite `addKV`.

Al momento della chiusura della votazione, il CEO invoca implicitamente `/api/v1/validation-vote`: il backend recupera l'intero storico della chiave di votazione tramite `getHistoryKV`, scarta i record di cancellazione (`isDelete`), somma algebricamente i valori ponderati di tutti i voti validi, conteggia le categorie (favorevoli, contrari, astenuti) e sigilla il risultato finale sulla blockchain con una nuova scrittura `addKV`. Il risultato è consultabile in sola lettura tramite `/api/v1/visualize-vote`.

Requisiti del Sistema

La seguente sezione descrive in modo sistematico e strutturato i requisiti del sistema UniTorV E-Vote, suddivisi in tre categorie principali: requisiti funzionali, requisiti non funzionali e requisiti strutturali/architetturali.

Requisiti Funzionali

Tabella 2: Requisiti funzionali

ID	Categoria	Descrizione
RF-01	Autenticazione	Il sistema deve permettere l'accesso tramite credenziali (wallet ID e password) con verifica lato server e generazione di token JWT.
RF-02	Autorizzazione	Il sistema deve differenziare i privilegi tra utenti CEO e Membri, limitando le operazioni sensibili ai soli CEO tramite middleware <code>isCeoOrAdmin</code> .
RF-03	Gestione Riunioni	I CEO devono poter creare riunioni specificando titolo, intervallo temporale, partecipanti e documento verbale PDF, quest'ultimo archiviato come Base64 sulla blockchain.
RF-04	Gestione Votazioni	Il sistema deve consentire l'apertura, la chiusura e lo scrutinio delle sessioni di voto all'interno di riunioni attive.
RF-05	Registrazione Voto	Ogni voto deve essere registrato sulla blockchain con valore ponderato per quota e identità del votante anonimizzata tramite SHA-256.
RF-06	Scrutinio	Al momento della chiusura di una votazione, il backend deve calcolare il risultato aggregato e sigillarlo sulla blockchain.
RF-07	Storico Riunioni	Il sistema deve esporre uno storico delle riunioni filtrato per ruolo (CEO: tutte; Membro: solo le proprie) con relativo stato.
RF-08	Download Verbale	Gli utenti autenticati devono poter scaricare il verbale PDF recuperandolo dalla blockchain e decodificandolo da Base64.
RF-09	Gestione Utenti	Il CEO deve poter visualizzare l'elenco completo degli utenti registrati (nome, email, wallet, ruolo) recuperandoli dalla blockchain.
RF-10	Notifica Riunione	Il sistema deve notificare i membri della presenza di una sessione attiva tramite popup nella dashboard.
Continua nella pagina successiva...		

Tabella 2 – Continua dalla pagina precedente

ID	Categoria	Descrizione
RF-11	Gestione Profilo	Gli utenti devono poter visualizzare i propri dati di profilo e disconnettersi in modo sicuro con pulizia del <code>localStorage</code> .

Requisiti Non Funzionali

Tabella 3: Requisiti non funzionali

ID	Categoria	Descrizione
RNF-01	Sicurezza – Autenticità	Le password devono essere memorizzate in forma hashed (bcrypt) sulla blockchain e mai in chiaro. La comunicazione client-server deve avvenire tramite HTTPS/TLS.
RNF-02	Sicurezza – Integrità	L'integrità dei voti e dei verbali deve essere garantita dall'immutabilità della blockchain: nessun record registrato può essere alterato retroattivamente.
RNF-03	Sicurezza – Confidenzialità	Il voto del singolo partecipante deve essere protetto tramite anonimizzazione SHA-256 dell'identità. Solo l'esito aggregato deve essere pubblicamente consultabile.
RNF-04	Sicurezza – Autorizzazione	Ogni endpoint API deve applicare controlli di accesso basati sul ruolo (RBAC). Le operazioni privilegiate devono richiedere un token JWT valido con claim CEO.
RNF-05	Disponibilità	Il sistema deve garantire una disponibilità minima del 99,5% durante i periodi di assemblea.
RNF-06	Scalabilità	L'architettura deve supportare un numero crescente di utenti e riunioni senza degrado delle prestazioni, sfruttando la struttura distribuita della blockchain.
RNF-07	Prestazioni	La risposta alle chiamate API di autenticazione e votazione deve avvenire entro 2 secondi in condizioni normali di carico. Il frontend applica un timeout di 15 secondi sulle operazioni di creazione riunione.
RNF-08	Tracciabilità	Ogni operazione critica (login, creazione riunione, voto, scrutinio) deve essere registrata in log di sistema con timestamp, ID utente ed esito, persistiti sulla blockchain.
Continua nella pagina successiva...		

Tabella 3 – Continua dalla pagina precedente

ID	Categoria	Descrizione
RNF-09	Anonimato del Voto	L'identità del votante non deve essere recuperabile in chiaro dal registro blockchain: solo l'hash SHA-256 dell'email deve essere presente nel payload.

Requisiti Strutturali e Architettureali

Tabella 4: Requisiti strutturali e architetturali

ID	Componente	Descrizione
RA-01	Frontend	La web application deve essere realizzata con tecnologie standard (HTML5, CSS3, JavaScript ES6+) e funzionare su tutti i browser moderni.
RA-02	Backend	Il server applicativo deve essere implementato in Node.js con Express.js, esporre una REST API versionata (/api/v1/) e applicare middleware di autenticazione per le rotte protette.
RA-03	Blockchain Layer	La registrazione di voti, riunioni, verbali e profili utente deve avvenire su un nodo blockchain dedicato (Tor Vergata Mainnet), garantendo immutabilità e verificabilità.
RA-04	File Storage	I verbali certificati in formato PDF devono essere codificati in Base64 e archiviati sulla blockchain nella classe Verbale , senza scrittura su filesystem locale.
RA-05	Separazione Ruoli	Il sistema deve implementare una separazione netta tra la dashboard CEO e la dashboard Membro, sia a livello UI che a livello API, tramite middleware isCeoOrAdmin .
RA-06	Modularità Backend	Il backend deve essere organizzato in file di routing separati per dominio funzionale: autenticazione, riunioni, votazioni, file, utenti, ruoli e KV store.

Architettura del Sistema

L'architettura del sistema UniTorV E-Vote è progettata secondo il paradigma client-server a più livelli (multi-tier architecture), con una separazione netta tra il livello di presentazione, il livello applicativo e il layer blockchain, che nella versione attuale assume il ruolo di unico sistema di persistenza.

Livello	Tecnologia	Responsabilità	Protocollo
Presentation Layer	HTML5, CSS3, JS ES6+	Interfaccia utente, interazione, rendering dashboard	HTTPS / TLS 1.3
Application Layer	Node.js + Express.js	Logica applicativa, autenticazione, routing API REST	HTTP/S REST API
Blockchain Layer	Tor Vergata Mainnet	Registrazione immutabile di voti, riunioni, verbali PDF (Base64) e profili utente	HTTP API (KV REST)

Tabella 5: Livelli architetturali del sistema

Architettura Client-Server e Separazione dei Livelli

Il sistema adotta una architettura client-server stratificata in tre livelli logicamente e fisicamente distinti, comunicanti tra loro esclusivamente attraverso interfacce ben definite.

Presentation Layer (Frontend)

Il livello di presentazione è implementato come una web application statica composta da pagine HTML5, fogli di stile CSS3 e script JavaScript ES6+. Il frontend non contiene logica di business né accede direttamente alla blockchain: ogni operazione che richieda elaborazione o accesso a dati sensibili viene delegata al backend tramite chiamate HTTP asincrone (fetch API con pattern `async/await`).

Il frontend è organizzato in viste dedicate: la landing page pubblica (`index.html`), la pagina di autenticazione (`login.html`), la dashboard CEO (`ceo_dashboard.html`), la dashboard Membro (`member_dashboard.html`), il pannello di creazione riunione (`create_meeting.html`), la pagina di dettaglio riunione (`meeting.html`), la sala di voto virtuale (`voting_room.html`), la pagina dei risultati (`results.html`), l'archivio storico (`audit_report.html`), la gestione utenti (`manage_users.html`) e le impostazioni del profilo (`settings.html`).

Il frontend implementa meccanismi di timeout sulle chiamate API (`AbortController` con scadenza configurabile) per prevenire situazioni di blocco indefinito. La verifica del ruolo utente per le pagine sensibili avviene tramite chiamata al server (`/api/v1/check-role`), rendendo il controllo di accesso indipendente dai dati memorizzati nel `localStorage`.

Application Layer (Backend)

Il livello applicativo è implementato tramite un server Node.js con framework Express.js. Il backend espone un'API RESTful versionata sotto il prefisso `/api/v1/`, organizzata in moduli di routing distinti per ciascun dominio funzionale:

- `login_route.js`: autenticazione utente e verifica token.
- `meetings_route.js`: recupero storico riunioni e creazione nuova riunione.
- `votation_route.js`: gestione stato votazioni, registrazione voti, scrutinio e visualizzazione risultati.
- `files_route.js`: upload e download dei verbali PDF tramite blockchain.
- `users_route.js`: recupero elenco utenti dalla blockchain (solo CEO).
- `role_route.js`: verifica del ruolo utente lato server.
- `mapping_route.js`: accesso diretto alle primitive del KV store blockchain (uso interno/debug).
- `logger_route.js`: esposizione dei log storici recuperati dalla blockchain.

Blockchain Layer (Tor Vergata Mainnet)

Il layer blockchain costituisce il componente più critico e il sistema di persistenza esclusivo della piattaforma. Il nodo Tor Vergata Mainnet espone un'interfaccia REST per la gestione di un Key-Value store distribuito e immutabile, tramite le funzioni `getKV`, `addKV`, `delKV`, `getHistoryKV`, `getKeysCopy` e `getNumKV` incapsulate nel modulo `mapping.js`.

Le classi principali utilizzate nel KV store sono:

- **Reunion**: metadati delle riunioni (chiave: `reunion_[timestamp]`).
- **Votation**: stato delle votazioni e payload dei singoli voti (chiave composta: `[meetingId, voteIndex]` oppure `[meetingId, "status"]`).
- **Verbale**: contenuto Base64 dei verbali PDF (chiave: `meetingId`).
- **Utenti**: profili utente con credenziali (hash `bcrypt`), ruolo e quota assembleare (chiave: `id_wallet`).
- **Logger**: log applicativi di tipo Alert e Signal (chiave: `Logger/Alert` e `Logger/Signal`).

La funzione `getHistoryKV` è utilizzata esclusivamente nel processo di scrutinio per recuperare tutti i record storici di una chiave di votazione, incluse le versioni precedenti sostituite: questo consente il conteggio completo di tutti i voti espressi anche dopo che la chiave è stata sovrascritta con il risultato sigillato.

Gestione dell'Autenticazione e delle Sessioni

Il sistema implementa un meccanismo di autenticazione stateless basato su JSON Web Token (JWT), conforme allo standard RFC 7519. Il flusso di autenticazione si articola nelle seguenti fasi: il client invia le credenziali al backend tramite HTTPS POST; il backend risolve l'identità dell'utente tramite la blockchain (mapping email → `id_wallet`, poi recupero dati completi); verifica la corrispondenza della password tramite `bcrypt.compare()`; genera un token JWT firmato con chiave segreta da variabile d'ambiente, con scadenza configurabile tramite `JWT_EXPIRES_IN`; restituisce il token e il profilo utente al client. Il client persiste il token nel `localStorage` e lo include nell'header `Authorization` (schema Bearer) di ogni successiva richiesta.

Nota di sicurezza: Il token JWT è validato ad ogni richiesta alle rotte protette dal middleware `verifyToken`. In caso di token scaduto o manomesso, il server risponde con HTTP 401, forzando il client a eseguire un nuovo ciclo di autenticazione. Il

`localStorage` viene pulito (`localStorage.clear()`) ad ogni logout o rilevazione di token non valido.

Comunicazione tramite API REST

Tutti i componenti del sistema comunicano esclusivamente attraverso un'API RESTful versionata. La tabella seguente elenca gli endpoint principali della versione attuale del sistema.

Tabella 6: Endpoint REST dell'API v1

Metodo	Endpoint	Auth	Funzione
POST	<code>/api/v1/loginCheck</code>	No (pubblica)	Autenticazione con password o verifica token JWT esistente.
GET	<code>/api/v1/check-role</code>	JWT	Verifica lato server se l'utente è CEO.
GET	<code>/api/v1/meetings</code>	JWT	Recupero storico riunioni filtrato per ruolo utente.
POST	<code>/api/v1/create-meeting</code>	JWT (CEO)	Creazione riunione e inizializzazione stati votazioni su blockchain.
GET	<code>/api/v1/get-votations-status</code>	JWT	Recupero stato (aperta/chiusa/non aperta) di tutte le votazioni di una riunione.
POST	<code>/api/v1/aggiorna-status</code>	JWT (CEO)	Aggiornamento stato delle votazioni di una riunione.
POST	<code>/api/v1/add-vote</code>	JWT	Registrazione voto ponderato e anonimizzato su blockchain.
POST	<code>/api/v1/validation-vote</code>	JWT (CEO)	Scrutinio della votazione e sigillo del risultato su blockchain.
POST	<code>/api/v1/visualize-vote</code>	JWT	Lettura dei risultati sigillati di una votazione.
POST	<code>/api/v1/uploadVerbale</code>	JWT (CEO)	Upload PDF verbale come Base64 su blockchain.
GET	<code>/api/v1/getVerbale/:id</code>	JWT	Download PDF verbale decodificato da Base64 dalla blockchain.
GET	<code>/api/v1/all-users</code>	JWT (CEO)	Recupero elenco utenti dalla blockchain.
POST	<code>/api/v1/addUser</code>	JWT (CEO)	Aggiunta di un nuovo utente alla blockchain.
GET	<code>/api/v1/logs/history</code>	(commentata)	Recupero log storici dalla blockchain (non attiva in produzione).

Componenti Architeturali

Frontend — Web Application

Il componente frontend è una web application multi-vista realizzata con tecnologie web standard. Al caricamento di ogni vista protetta, il JavaScript esegue una verifica preliminare della presenza del token JWT nel `localStorage`: in assenza di token, l'utente viene reindirizzato alla pagina di login. Per le pagine che richiedono verifica del ruolo (es. `meeting.html`, `manage_users.html`), la verifica avviene tramite chiamata al server all'endpoint `/api/v1/check-role`.

La Dashboard CEO presenta: il pannello di creazione riunione, la gestione degli stati delle votazioni (apertura/chiusura), l'accesso all'audit report completo, il pannello di gestione utenti e il popup di notifica per riunioni attive. La Dashboard Membro mostra: le riunioni a cui l'utente è invitato (filtrate per email), il popup di notifica per sessioni attive, l'accesso alle sale di voto per le votazioni aperte e la visualizzazione dei risultati per le votazioni concluse.

6.1.1 Sala di Voto Virtuale (`voting_room.js`)

La sala di voto esegue prima del caricamento una verifica esplicita dello stato della votazione tramite `/api/v1/get-votations-status`: se la votazione non è nello stato `true`, l'accesso viene bloccato anche se il link è stato raggiunto direttamente via URL, costituendo un meccanismo di controllo di accesso lato client complementare a quello lato server.

6.1.2 Pagina dei Risultati (`results.js`)

I risultati vengono letti dal `localStorage` (dove sono stati salvati dalla chiamata a `/api/v1/visualize-vote`) e presentati con visualizzazione a barre percentuali per favorevoli, contrari e astenuti, calcolo dell'affluenza rispetto ai partecipanti aventi diritto, somma algebrica delle quote e badge di esito (approvata/respinta/pareggio).

Backend Server — Node.js con Express.js

6.2.1 Gestione dell'Autenticazione (`login_route.js`)

Il modulo di autenticazione si avvale del middleware `fetchUserFromBlockchain` per la separazione delle responsabilità tra il recupero dei dati dalla blockchain e la logica di verifica delle credenziali. L'endpoint `/api/v1/loginCheck` gestisce due casi: la presenza di un token JWT nell'header `Authorization` (verifica diretta tramite `jwt.verify()`) e il login tradizionale con password (confronto tramite `bcrypt.compare()`). Il processo segue il principio di *fail-safe defaults*: in assenza di corrispondenza, il sistema nega l'accesso con HTTP 401 e un messaggio generico, senza specificare la natura dell'errore.

6.2.2 Gestione delle Riunioni (`meetings_route.js`)

Il modulo espone due endpoint. L'endpoint GET `/api/v1/meetings` esegue lato server l'intera logica di aggregazione dello storico: recupera le chiavi dalla blockchain, carica i payload, filtra per ruolo e ordina per timestamp. Questo approccio centralizza la logica di business, migliorando la sicurezza (il filtraggio per email è ora server-side e non aggirabile) e le prestazioni (meno round-trip HTTP). L'endpoint POST `/api/v1/create-meeting` gestisce la creazione della riunione e l'inizializzazione automatica degli stati delle votazioni.

6.2.3 Gestione delle Votazioni (`votation_route.js`)

Questo modulo implementa l'intero ciclo di vita di una votazione. L'endpoint `POST /api/v1/add-vote` applica la catena di middleware `verifyToken` + `checkIfAlreadyVoted`, recupera la quota dell'utente dalla blockchain tramite `fetchUserFromBlockchain`, calcola il valore ponderato, anonimizza l'email tramite `crypto.createHash("sha256")` e registra il payload sulla blockchain. L'endpoint `POST /api/v1/validation-vote` recupera lo storico completo tramite `getHistoryKV`, filtra i record di cancellazione, aggrega i risultati e li sigilla con `addKV`. L'endpoint `POST /api/v1/visualize-vote` verifica che il payload della votazione contenga il campo `dettagliVoti` (presente solo dopo la validazione) prima di restituire i risultati.

6.2.4 Gestione dei Verbali PDF (`files_route.js`)

Il middleware `Multer` è configurato con `memoryStorage`: il file PDF non viene mai scritto su disco. Il buffer binario ricevuto viene convertito in stringa Base64 tramite `buffer.toString("base64")` e salvato sulla blockchain con classe `Verbale`. In lettura, la stringa Base64 viene recuperata dalla blockchain, convertita in `Buffer` tramite `Buffer.from(base64Data, "base64")` e inviata al client con header `Content-Type: application/pdf` e `Content-Disposition: inline`.

6.2.5 Gestione Utenti (`users_route.js`)

Il modulo espone due endpoint. L'endpoint `GET /api/v1/all-users` recupera l'elenco completo degli utenti direttamente dalla blockchain, iterando sulle chiavi della classe `Utenti`. L'endpoint `POST /api/v1/addUser` inserisce un nuovo utente sulla blockchain, dopo aver verificato che l'email non sia già presente nella tabella di mapping. In entrambi i casi, l'hash bcrypt della password non viene mai esposto nella risposta HTTP.

6.2.6 Middleware e Cross-Cutting Concerns

Il backend applica middleware trasversali: `verifyToken` (verifica JWT su tutte le rotte protette), `isCeoOrAdmin` (verifica del ruolo CEO su operazioni privilegiate), `checkIfAlreadyVoted` (prevenzione del double voting), `fetchUserFromBlockchain` (risoluzione identità dalla blockchain), `CORS` (controllo origini autorizzate), `express.json()` (parsing del body JSON). La gestione degli errori è strutturata per differenziare errori di rete, di autorizzazione e applicativi, restituendo codici HTTP appropriati (400, 401, 403, 404, 500).

Sistema di Logging Applicativo

Architettura del Logger

Il backend dell'applicazione UniTorV E-Vote implementa un sistema di logging centralizzato basato sulla classe `Logger` (modulo `logger_utils.js`). Ogni evento rilevante – sia esso un accesso autorizzato, un tentativo fallito o un errore di sistema – viene registrato direttamente sulla blockchain, garantendo immutabilità e tracciabilità dell'audit trail.

Il logger distingue due tipologie di evento:

- **Alert** – eventi anomali, errori di autenticazione, accessi negati, eccezioni non gestite. Vengono salvati sulla blockchain con chiave `Logger/Alert`.
- **Signal** – operazioni legittime completate con successo. Vengono salvati con chiave `Logger/Signal`.

Formato del log

Ogni voce di log è una stringa con il seguente formato:

```
[ [<timestamp> ] [ <ip-sorgente> ] [ <ip-destinazione> ] [ <method> <route> ] [ <codice HTTP> ] [ <commento> ] ]
```

Esempio reale generato dal sistema:

```
[ [2026-03-15T10:45:32.001Z] [192.168.1.42] [10.0.0.1] [POST /api/v1/loginCheck] [401] [Login fallito per identificativo: user@example.com. Password errata.] ]
```

I log vengono inoltre stampati in console per il debug locale:

- Alert → `console.error()` con prefisso `[BLOCKCHAIN ALERT SALVATO]`
- Signal → `console.log()` con prefisso `[BLOCKCHAIN SIGNAL SALVATO]`

Recupero dei Log

La route `GET /api/v1/logs/history` (modulo `logger_route.js`) espone i log storici recuperando dallo storico blockchain entrambe le chiavi (`Alert` e `Signal`). Il parsing avviene estraendo il campo `value` da ogni record non cancellato (`isDelete = false`) e filtrando le stringhe che iniziano con `[`.

Nota di sicurezza: La route `/logs/history` è attualmente **commentata** nel file `server.js`, il che la rende inaccessibile in produzione. Questo comportamento è intenzionale o una misura temporanea da valutare prima del rilascio.

Mappatura degli Eventi di Log per Route

Nelle sezioni seguenti viene documentato sistematicamente ogni evento di log emesso da ciascuna route del backend, con la relativa classificazione (*Alert* o *Signal*), il codice HTTP associato e la descrizione del contesto.

Autenticazione – login_route.js

Route	Tipo	HTTP	Descrizione
POST /loginCheck	Signal	200	Richiesta di login manuale ricevuta per l'identificativo fornito.
POST /loginCheck	Alert	400	Login fallito: password assente nel corpo della richiesta.
POST /loginCheck	Signal	200	Login completato con successo; token JWT generato.
POST /loginCheck	Alert	401	Credenziali errate: password non corrispondente.
POST /loginCheck	Alert	500	Errore interno durante l'elaborazione del login.

Tabella 7: Log events – login_route.js

Middleware di Autenticazione – auth_middle.js

Funzione	Tipo	HTTP	Descrizione
verifyToken	Alert	401	Token JWT assente nell'header <code>Authorization</code> .
verifyToken	Alert	403	Token presente ma non valido o scaduto.
isCeoOrAdmin	Alert	403	Dati utente o campo <code>classe</code> assenti nel payload.
isCeoOrAdmin	Alert	403	Accesso negato per ruolo insufficiente.
quickTokenCheck	Signal	200	Token verificato con successo (bypass blockchain).
quickTokenCheck	Alert	401	Token scaduto o non valido nel percorso di bypass.
quickTokenCheck	Signal	200	Nessun token fornito; proseguimento verso blockchain.

Tabella 8: Log events – auth_middle.js

Middleware Recupero Utente – `fetch_user_middle.js`

Funzione	Tipo	HTTP	Descrizione
<code>fetchUserFromBlockchain</code>	Alert	500	Errore di parsing tabella email/wallet.
<code>fetchUserFromBlockchain</code>	Alert	404	Email non presente nella tabella di mapping.
<code>fetchUserFromBlockchain</code>	Signal	200	Risoluzione identità riuscita: email → id_wallet.
<code>fetchUserFromBlockchain</code>	Alert	400	Nessun identificativo fornito (wallet/email).
<code>fetchUserFromBlockchain</code>	Alert	404	Nessun dato trovato per l'id_wallet fornito.
<code>fetchUserFromBlockchain</code>	Alert	500	Errore di parsing dei dati utente.
<code>fetchUserFromBlockchain</code>	Signal	200	Dati utente recuperati con successo.
<code>fetchUserFromBlockchain</code>	Alert	500	Eccezione generica nel middleware.

Tabella 9: Log events – `fetch_user_middle.js`Middleware Verifica Voto – `check_vote_middle.js`

Funzione	Tipo	HTTP	Descrizione
<code>checkIfAlreadyVoted</code>	Alert	400	Parametri <code>meetingId</code> o <code>voteIndex</code> assenti.
<code>checkIfAlreadyVoted</code>	Signal	200	Nessuno storico trovato: prima votazione consentita.
<code>checkIfAlreadyVoted</code>	Signal	200	Storico vuoto: voto consentito.
<code>checkIfAlreadyVoted</code>	Alert	403	Voto duplicato: hash utente già presente nello storico.
<code>checkIfAlreadyVoted</code>	Signal	200	Analisi completa: nessun voto precedente rilevato.
<code>checkIfAlreadyVoted</code>	Alert	500	Eccezione generica nel middleware.

Gestione Riunioni – `meetings_route.js`

Route	Tipo	HTTP	Descrizione
GET <code>/meetings</code>	Alert	400	Email utente assente nel token (utenti non-CEO).
GET <code>/meetings</code>	Alert	500	Errore nel recupero di una singola riunione.
GET <code>/meetings</code>	Signal	200	Riunioni recuperate con successo.
GET <code>/meetings</code>	Alert	500	Errore generale nel recupero delle riunioni.
POST <code>/create-meeting</code>	Alert	400	Parametri obbligatori assenti nel corpo richiesta.
POST <code>/create-meeting</code>	Signal	200	Riunione e votazioni create con successo.
POST <code>/create-meeting</code>	Alert	500	Errore durante la creazione su blockchain.

Gestione File – `files_route.js`

Route	Tipo	HTTP	Descrizione
POST /uploadVerbale	Alert	400	Upload fallito: file PDF o meetingId assente.
POST /uploadVerbale	Signal	200	Verbale caricato con successo sulla blockchain.
POST /uploadVerbale	Alert	500	Errore durante l'upload del verbale.
GET /getVerbale/:id	Alert	400	Parametro <code>id_reunion</code> assente.
GET /getVerbale/:id	Alert	404	Verbale non trovato per l'id fornito.
GET /getVerbale/:id	Alert	500	Dati del verbale non in formato stringa valido.
GET /getVerbale/:id	Signal	200	Verbale recuperato e inviato correttamente.
GET /getVerbale/:id	Alert	500	Errore tecnico durante il recupero.

Gestione Utenti – `users_route.js`

Route	Tipo	HTTP	Descrizione
GET /all-users	Alert	500	Errore di parsing o recupero utente specifico.
GET /all-users	Signal	200	Lista utenti recuperata con successo.
GET /all-users	Alert	500	Errore generale lettura utenti da blockchain.
POST /addUser	Alert	400	Registrazione fallita: campi obbligatori mancanti.
POST /addUser	Alert	400	Email già in uso (tentativo duplicato).
POST /addUser	Signal	200	Utente aggiunto con successo alla blockchain.
POST /addUser	Alert	500	Errore durante l'inserimento del nuovo utente.

Verifica Ruolo – `role_route.js`

Route	Tipo	HTTP	Descrizione
GET /check-role	Signal	200	Dati assenti nel token; restituito <code>isCEO: false</code> .
GET /check-role	Signal	200	Verifica ruolo completata correttamente.
GET /check-role	Alert	500	Errore interno durante la verifica del ruolo.

Riepilogo Statistico degli Eventi di Log

Modulo	Alert	Signal	Totale
login_route.js	3	2	5
auth_middle.js	4	3	7
fetch_user_middle.js	5	3	8
check_vote_middle.js	3	3	6
meetings_route.js	4	2	6
files_route.js	5	2	7
users_route.js	5	2	7
role_route.js	1	2	3
Totale	30	19	49

Tabella 10: Distribuzione degli eventi di log per modulo backend.

Analisi di Sicurezza del Sistema di Logging

Punti di Forza

1. **Immutabilità tramite blockchain.** Ogni log viene scritto sulla blockchain con `addKV("Logger", type, logBody)`, rendendo estremamente difficile la manomissione retroattiva dell'audit trail.
2. **Tracciabilità completa delle identità.** Gli Alert includono sistematicamente wallet, email (o hash SHA-256 nel caso di `check_vote_middle.js`) e ruolo, permettendo di correlare ogni evento anomalo a un soggetto specifico.
3. **Logging dei tentativi di accesso non autorizzato.** Il middleware `isCeoOrAdmin` registra ogni tentativo di accesso a risorse privilegiate da parte di utenti con ruolo insufficiente, includendo wallet e ruolo effettivo.
4. **Rilevamento del double voting.** Il middleware `checkIfAlreadyVoted` genera un Alert specifico (HTTP 403) quando rileva un tentativo di voto duplicato, includendo l'hash dell'utente e gli identificativi della votazione.

Threat Modeling — Modello STRIDE

Il Threat Modeling è il processo sistematico di identificazione, classificazione e valutazione delle minacce potenziali che possono compromettere la sicurezza di un sistema software. Il modello STRIDE, sviluppato da Microsoft, fornisce un framework strutturato per la categorizzazione delle minacce in sei classi fondamentali.

Spoofing — Impersonificazione

Gli attacchi di Spoofing mirano a far sì che un attaccante possa presentarsi al sistema con un'identità diversa dalla propria. Nel contesto di UniTorV E-Vote, il vettore di Spoofing più rilevante è rappresentato dal sistema di autenticazione: la compromissione delle credenziali di un utente CEO consentirebbe all'attaccante di assumere il pieno controllo amministrativo della piattaforma, creando riunioni fraudolente, manipolando stati delle votazioni e accedendo ai verbali riservati.

Un secondo scenario riguarda l'impersonificazione del nodo blockchain: un attaccante che controlli il percorso di rete tra il backend e il nodo Tor Vergata Mainnet potrebbe simulare risposte contraffatte del registro distribuito, inducendo il backend ad accettare uno stato falso del sistema. La natura stateless del JWT aggiunge un ulteriore vettore: la compromissione della chiave segreta `JWT_SECRET` consentirebbe la generazione di token arbitrariamente validi con qualsiasi ruolo.

Tampering — Manipolazione dei Dati

Gli attacchi di Tampering mirano alla modifica non autorizzata dei dati del sistema. Il vettore più critico è la manipolazione dei voti: un attaccante che intercetti la comunicazione tra frontend e backend (in assenza di TLS) potrebbe alterare il payload di voto, modificando il valore di `voto` (da "contrario" a "favorevole") o il `meetingId`. L'adozione della blockchain mitiga questo rischio per i dati già registrati.

Un vettore aggiuntivo riguarda la manipolazione dei verbali PDF sulla blockchain: poiché il PDF è memorizzato come stringa Base64 nel KV store, un attaccante con capacità di scrittura sulla blockchain (token CEO compromesso) potrebbe sovrascrivere la stringa Base64 con un documento contraffatto.

Repudiation — Ripudio delle Azioni

La Repudiation si verifica quando un utente nega di aver compiuto un'azione nel sistema. In un sistema di voto elettronico aziendale, un membro potrebbe contestare la validità della propria partecipazione a una votazione. La protezione contro la Repudiation è realizzata dall'associazione crittografica tra ogni voto e l'hash SHA-256 dell'email del votante nel registro blockchain immutabile. Tuttavia, poiché l'hash è non-invertibile (a meno di attacchi a dizionario), la risoluzione di dispute legali richiede che il sistema disponga di un meccanismo di verifica controllata dell'identità (il sistema sa se un'email produce un determinato hash, senza rivelare l'identità a terzi).

Information Disclosure — Divulgazione di Informazioni

Il vettore più critico è la compromissione delle credenziali: poiché i profili utente sono registrati sulla blockchain, un attaccante con accesso di lettura al KV store potrebbe recuperare i dati degli utenti inclusi gli hash bcrypt. Tuttavia, la resistenza computazionale di bcrypt mitiga il rischio di

inversione. Un secondo vettore è rappresentato dagli attacchi XSS sul frontend: se i dati restituiti dal backend vengono inseriti nell'HTML senza sanificazione, un attaccante potrebbe iniettare script malevoli che leggono il contenuto del `localStorage`, ottenendo il token JWT. L'endpoint `/api/v1/all-users` aggiunge un vettore di Information Disclosure se non adeguatamente protetto: la risposta include email, wallet e ruolo di tutti gli utenti del sistema.

Denial of Service — Interruzione del Servizio

Un attacco DoS durante una sessione di voto attiva è particolarmente grave, potendo impedire ai membri di esprimere il proprio voto entro i termini previsti. Il backend Node.js è sensibile agli attacchi di application-layer DoS: operazioni computazionalmente costose (come la verifica bcrypt di un numero elevato di richieste di login fraudolente) possono saturare il thread principale. L'endpoint `/api/v1/uploadVerbale`, che gestisce file fino a 50MB, è un vettore specifico di DoS per esaurimento della memoria: l'utilizzo di `memoryStorage` in Multer implica che file di grandi dimensioni vengano interamente caricati in RAM prima dell'elaborazione.

Elevation of Privilege — Escalation dei Privilegi

Il confine di autorizzazione più critico è quello tra il ruolo CEO e il ruolo Membro. Il principale vettore è la compromissione della chiave segreta JWT: con questa chiave, un attaccante potrebbe generare token con il campo `classe` impostato a "CEO", bypassando l'intero sistema di autenticazione. Nella versione attuale, la verifica server-side tramite `/api/v1/check-role` aggiunge un livello di controllo, ma dipende anch'essa dalla validità del token JWT.

Attack Surface Analysis

L'Attack Surface Analysis identifica tutti i punti attraverso cui un attaccante potrebbe tentare di compromettere il sistema.

Tabella 11: Attack Surface Analysis — vettori identificati

ID	Vettore	Descrizione	STRIDE	Rischio
A-01	Interfaccia Web (Login Form)	Form di login accessibile pubblicamente. Esposto a credential stuffing e brute force.	Spoofing, Info Disclosure	CRITICO
A-02	/api/v1/loginCheck	Unico endpoint pubblico del sistema. Assenza di rate limiting espone ad attacchi di forza bruta automatizzati.	Spoofing, DoS	CRITICO
A-03	/api/v1/add-vote	Endpoint di voto. Un token compromesso consente voti fraudolenti ponderati per quota.	Tampering, Repudiation	ALTO
A-04	/api/v1/uploadVerbaLe	Endpoint di upload file con memoryStorage. File da 50MB caricati in RAM; rischio DoS per esaurimento memoria.	Tampering, DoS	ALTO
A-05	/api/v1/all-users	Endpoint che restituisce email, wallet e ruolo di tutti gli utenti dalla blockchain. Esposto a Information Disclosure se il token CEO viene compromesso.	Info Disclosure	ALTO
A-06	/api/v1/aggiorna-stato	Aggiornamento stato votazioni. Un CEO malintenzionato o con token compromesso può aprire/chiudere votazioni in modo fraudolento.	Tampering, EoP	ALTO
Continua nella pagina successiva...				

Tabella 11 – Continua dalla pagina precedente

ID	Vettore	Descrizione	STRIDE	Rischio
A-07	LocalStorage (Frontend)	Storage del token JWT e dei dati utente nel browser. Esposto a lettura tramite XSS se il frontend non sanifica i dati in output.	Spoofing, Info Disclosure	ALTO
A-08	Canale Backend-Blockchain	Comunicazione HTTP verso il nodo blockchain. Assenza di autenticazione mutua espone a impersonificazione del nodo.	Spoofing, Tampering	ALTO
A-09	Variabili d'Ambiente (.env)	Chiave segreta JWT e configurazioni sensibili. Esposizione tramite repository non protetti o logging accidentale.	Spoofing, EoP	CRITICO
A-10	Payload Base64 Verbal	Stringhe Base64 dei verbali PDF sulla blockchain. Chiunque abbia accesso di lettura al KV store può scaricare i documenti.	Info Disclosure	MEDIO
A-11	Mapping Route (debug)	Endpoint <code>/api/v1/getKv</code> , <code>addKv</code> ecc. privi di autenticazione JWT. Consentono lettura/scrittura arbitraria del KV store.	Tampering, Info Disclosure	CRITICO
A-12	<code>/api/v1/logs/history</code>	Route di accesso ai log storici attualmente disabilitata ma priva di autenticazione. Se riabilitata, espone l'intero audit trail.	Info Disclosure	ALTO

Autenticazione e Gestione delle Sessioni

Il `localStorage` del browser, utilizzato per la persistenza del token JWT, è intrinsecamente vulnerabile agli attacchi XSS: qualsiasi script malevolo iniettato nelle pagine del frontend può leggere il contenuto del `localStorage` e trasmettere il token a un server controllato dall'attaccante.

L'alternativa più sicura è l'utilizzo di cookie `HttpOnly` per la trasmissione del token, non accessibili via JavaScript.

Comunicazioni e Canali di Rete

Il canale di comunicazione tra backend e nodo blockchain è vulnerabile all'impersonificazione del nodo in assenza di meccanismi di autenticazione mutua. Poiché l'intera persistenza del sistema transita per questo canale, la sua protezione rappresenta una priorità assoluta.

Principio chiave: La superficie di attacco di un sistema è direttamente proporzionale al numero di interfacce esposte e alla quantità di input accettata da ciascuna. Ridurre la superficie di attacco significa limitare gli endpoint pubblici al minimo indispensabile, applicare il principio del minimo privilegio e validare rigorosamente ogni input prima di qualsiasi elaborazione.

Mitigation Strategies

Le strategie di mitigazione definiscono l'insieme delle contromisure tecniche e organizzative adottate o raccomandate per ridurre la probabilità di successo degli attacchi identificati e limitare l'impatto di eventuali compromissioni.

Autenticazione Sicura e Protezione delle Credenziali

La protezione delle credenziali degli utenti è implementata attraverso l'hashing bcrypt con fattore di lavoro configurabile, che garantisce resistenza agli attacchi di forza bruta offline. I dati risultanti vengono persistiti esclusivamente sulla blockchain, eliminando la dipendenza da un database relazionale e sfruttando l'immutabilità del registro distribuito. Il rate limiting sull'endpoint di login rappresenta la contromisura principale contro gli attacchi di brute force e credential stuffing, implementabile tramite middleware `express-rate-limit` con soglie di richieste per indirizzo IP.

La gestione della chiave segreta JWT tramite variabile d'ambiente (`JWT_SECRET`) e mai hardcoded nel codice sorgente costituisce una misura fondamentale per prevenire la generazione di token fraudolenti. La rotazione periodica della chiave JWT è raccomandata, con un meccanismo di grace period per la gestione dei token emessi prima della rotazione.

Protezione delle Comunicazioni

L'adozione di HTTPS con TLS 1.3 su tutte le comunicazioni frontend-backend è il prerequisito fondamentale per la protezione dei dati in transito. La **Content Security Policy (CSP)** limita le sorgenti da cui il browser può caricare risorse, riducendo significativamente la superficie di attacco per le vulnerabilità XSS e rendendo molto più difficile il furto del token JWT dal `localStorage`.

La gestione del file size limit (50MB) in Multer deve essere accompagnata da una validazione esplicita del tipo MIME lato server: il solo controllo del Content-Type dichiarato dal client non è sufficiente, è necessario verificare la firma binaria (magic bytes) del file ricevuto per garantire che sia effettivamente un PDF.

Integrità tramite Blockchain e Controllo degli Accessi

La blockchain è il meccanismo primario di garanzia dell'integrità dell'intero sistema: voti, riunioni, verbali e profili utente sono tutti registrati in modo immutabile. Il meccanismo di prevenzione del *double voting*, implementato tramite il middleware `checkIfAlreadyVoted`, garantisce che ogni partecipante possa esprimere al massimo un voto per sessione. L'anonimizzazione dell'identità tramite SHA-256 protegge la privacy del votante, rendendo il registro blockchain pubblicamente verificabile senza rivelare l'associazione tra identità e scelta di voto.

Il controllo degli accessi basato sui ruoli (RBAC), implementato tramite middleware di autorizzazione nel backend, garantisce che ogni operazione del sistema sia accessibile esclusivamente agli utenti con i privilegi appropriati. La verifica del ruolo avviene sia a livello di routing (middleware `isCeoOrAdmin`) sia a livello di logica applicativa, seguendo il principio di Defense in Depth.

Logging, Auditing e Monitoraggio

Il sistema implementa un logging applicativo completo basato sulla classe `Logger`: ogni evento critico viene registrato direttamente sulla blockchain con tipologia Alert o Signal, garantendo

immutabilità dell'audit trail. I log includono sistematicamente timestamp ISO 8601, indirizzi IP sorgente e destinazione, metodo e route HTTP, codice di risposta e messaggio descrittivo.

L'implementazione di alert automatici per anomalie rilevabili (numero di tentativi di login falliti superiore alla soglia, voti multipli dallo stesso IP, apertura/chiusura rapida di votazioni) consente una risposta proattiva agli incidenti. La route di accesso ai log (`/logs/history`) deve essere protetta tramite autenticazione CEO prima di essere abilitata in produzione.

Considerazione conclusiva: L'analisi STRIDE e la definizione delle strategie di mitigazione evidenziano come la sicurezza di UniTorV E-Vote richieda un approccio stratificato. La blockchain garantisce l'integrità, il non-ripudio e la tracciabilità di voti, verbali e profili utente; HTTPS e JWT garantiscono la confidenzialità delle comunicazioni e l'autenticità delle sessioni; RBAC e rate limiting limitano la superficie di attacco applicativa; bcrypt protegge le credenziali anche in caso di lettura non autorizzata del KV store. La chiusura della vulnerabilità degli endpoint di mapping e la protezione della route dei log rappresentano le priorità tecniche più urgenti per raggiungere un livello di sicurezza adeguato a un sistema di voto elettronico aziendale.

Definizione dell'infrastruttura

Definizione delle Zone (IEC 62443)

L'architettura di rete è stata segmentata seguendo i principi fondamentali dello standard internazionale IEC 62443. Questa suddivisione in zone di sicurezza e "conduit" (condotti di comunicazione) permette di isolare i sistemi critici, limitare i movimenti laterali di un eventuale attaccante in caso di compromissione e applicare policy di accesso granulari basate sul principio del minimo privilegio.

Zona IT / Enterprise

Componente	Descrizione
Portale Web	Portale principale per la gestione delle riunioni e dei dati amministrativi relativi a quest'ultime
Server Web	Server che forniscono servizi interni comunicazione e gestione utenti
Blockchain	Blockchain contenente i dati del personale e delle riunioni.
Wi-fi	Tramite per la connessione privata al sistema.
Active directory	Server con il compito di autenticare e autorizzare utenti, computer e risorse

Tabella 12: Componenti della zona IT / Enterprise

Zona DMZ

Componente	Descrizione
Proxy / Gateway	Software intermediario che agisce per conto di un utente, fungendo da gateway tra il Front-end e i server interni.
Server di patch	Server che scarica e controlla gli aggiornamenti che eventualmente verranno scaricati nella zona OT.
Jump server	Server che aprono una connessione sicura per gli elementi della zona OT.

Tabella 13: Componenti della zona DMZ

Zona OT / Site

Componente	Descrizione
Server SCADA	Server di monitoraggio, acquisizioni dati e controllo processi aziendali.

Tabella 14: Componenti della zona OT / Site

Zona Field

Componente	Descrizione
Dispositivi aziendali	Tablet con software integrato e tracciato

Tabella 15: Componenti della zona Field

Asset Inventory

Il primo passo fondamentale per un'efficace modellazione delle minacce è l'identificazione e la classificazione degli asset. In questa sezione vengono catalogati i componenti hardware, software e di rete dell'infrastruttura, assegnando a ciascuno un livello di criticità in base all'impatto di una potenziale violazione (in termini di disponibilità, integrità o riservatezza dei dati medici e personali). Vengono inoltre mappate le dipendenze tecnologiche, essenziali per comprendere gli effetti a catena di un eventuale disservizio.

- Portale Web
- Server Web
- Wi-Fi aziendale
- Server SCADA
- Blockchain
- Proxy
- Jump server
- Active directory
- Server di Patch
- Tablet

Asset	Tipo	Criticità	Dipendenze
Portale Web	Applicazione Web	Critica	Server Web
Server Web	Server IT	Alta	Intranet, directory servizi
Wi-Fi	Rete wireless	Media	Internet, AAA
Server SCADA	Server	Critica	Firewall, Intranet
Blockchain	Database	Critica	Software Web
Proxy	Server	Alto	Intranet
Jump server	Server	Alta	Intranet
Active directory	Database	Crititca	Intranet (?)
Tablet	OT	Critica	Intranet

Tabella 16: Inventario degli asset

Superficie Osservabile

La superficie osservabile rappresenta ciò che un potenziale attaccante o un analista può dedurre dall'esterno semplicemente intercettando il traffico di rete o analizzando i metadati delle comunicazioni, senza avere accesso diretto al payload dei pacchetti o alle credenziali applicative. Mappare questi elementi è cruciale per comprendere quali informazioni l'infrastruttura espone involontariamente e come queste possano essere sfruttate per la successiva fase di enumerazione dei target.

Asset	Cosa osservo	Cosa inferisco
Portale web	Numero accessi al portale	Normale attività del servizio
Server web	Volume di traffico e numero di connessioni	Normale attività dei servizi
Wi-Fi Aziendale	Numero di dispositivi connessi	Presenza di dispositivi connessi
Server SCADA	Traffico di rete su porte specifiche	Normale attività dei servizi
Blockchain	Frequenza delle letture e scritture	Accesso ai dati del sistema
Proxy	Connessione in uscita a domini specifici	Normale attività del server Proxy
Jump server	Connessione remota dall'esterno	Richiesta dati della zona OT dalla zona IT
Active directory	Volume di traffico e numero di connessioni	Normale attività dei servizi
Tablet	Comunicazioni periodiche con server	Normale funzionamento dei dispositivi

Tabella 17: Superficie osservabile degli asset

Grafo dell'Infrastruttura

Per fornire una visione olistica dell'architettura e agevolare l'analisi dei rischi, viene di seguito presentato il grafo dell'infrastruttura di rete. Questo schema evidenzia visivamente la segmentazione in zone, i nodi critici, i percorsi di comunicazione consentiti (con i relativi protocolli impiegati) e la frequenza stimata delle interazioni, facilitando l'identificazione di eventuali colli di bottiglia o vulnerabilità nei punti di interscambio.

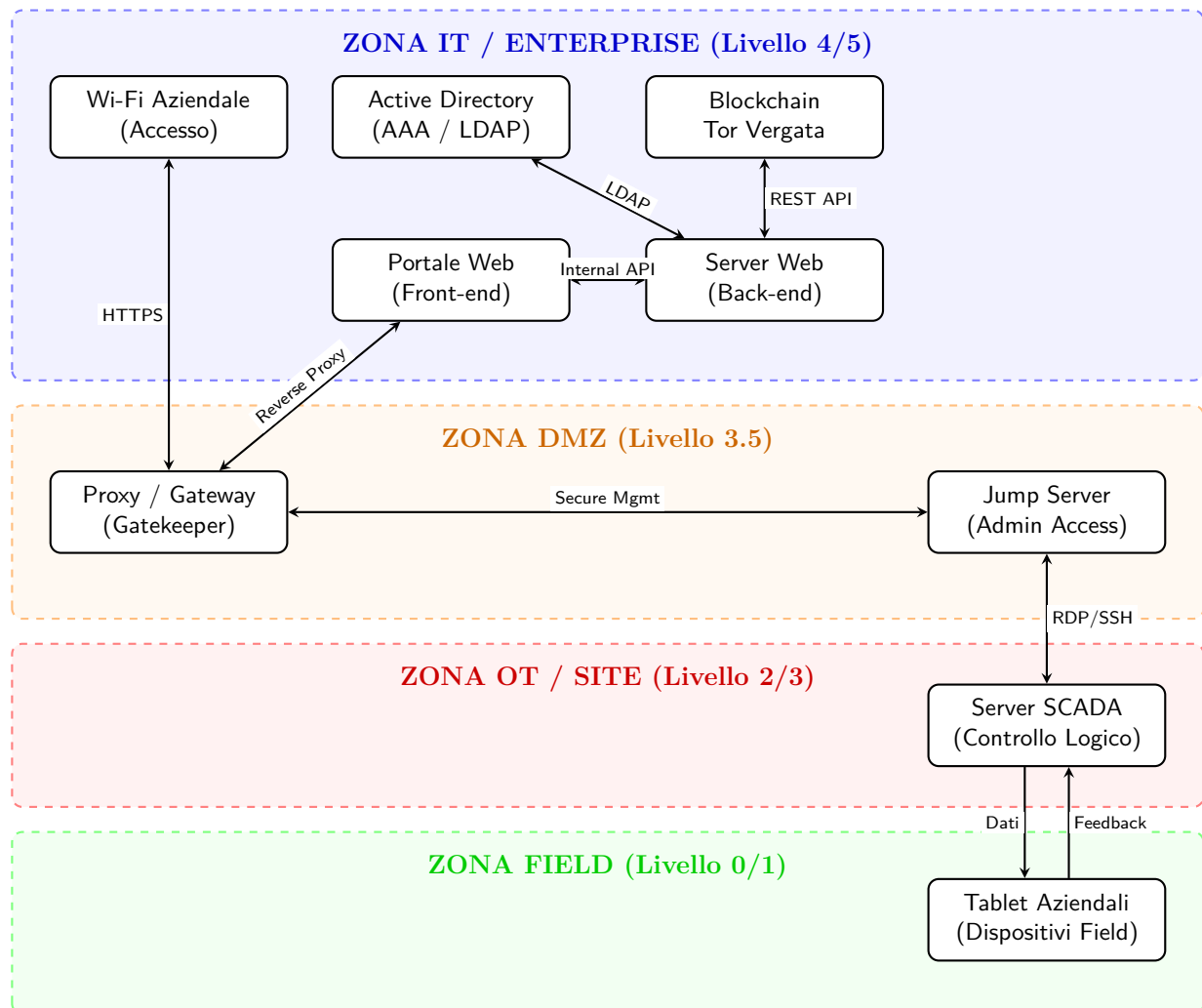


Figura 1: Infrastruttura UniTorV E-Vote: Architettura espansa orizzontalmente

Definizione della Baseline e Analisi delle Deviazioni

Prima di analizzare gli eventi osservati, è necessario definire l'involuppo operativo (*Operating Envelope*) del sistema. Basandoci sull'architettura segmentata IEC 62443 definita nel Threat Model, stabiliamo quali comunicazioni sono legittime considerando vincoli temporali, volumetrici e topologici. Questa *baseline* rappresenta la regione dei comportamenti ammessi: ogni evento che fuoriesce da questi confini costituirà una deviazione da indagare.

Definiamo ora le relazioni più significative:

Relazione	Normale	Quando	Intensità	Note
Portale web → Server web	Sì	Sempre	Alta	Normale funzionamento del sistema
Server web → Blockchain	Sì	Sempre	Alta	Backend applicativo
Wi-fi → Proxy	Sì	Sempre	Alta	Prova di connessione al sistema
Proxy → Portale web	Sì	Sempre	Alta	Accesso al sistema
Server web → Active Directory	Sì	Sempre	Media	Autenticazione da parte di un'utente
Proxy → Jump server	Sì	Manutenzione	Bassa	Invio aggiornamenti da parte dei sistemisti ¹
Jump server → Proxy	Sì	Periodica	Media	Invio dati riunioni raccolti dai tablet
Jump server → Server Scada	Sì	Manutenzione	Bassa	Accesso da parte dei sistemisti
Server Scada → Jump server	Sì	Periodica	Media	Invio dati riunioni raccolti dai tablet
Server Scada → Tablet	Sì	Manutenzione	Bassa	Scaricamento degli aggiornamenti
Tablet → Server Scada	Sì	Periodica	Media	Invio feedback
Wi-fi → Jump server	No	Mai	Nulla	Deviazione topologica
Wi-fi → Blockchain	No	Mai	Nulla	Percorso non previsto

Tabella 18: Baseline comportamentale delle relazioni significative.

Valutazione del Dataset Osservato

Step 3: Dati Grezzi

La tabella seguente riporta il traffico di rete grezzo campionato in una determinata finestra temporale. L'obiettivo non è cercare un attacco in astratto, ma confrontare questi log con la baseline appena definita per identificare le deviazioni nell'architettura UniTorV E-Vote.

Time	Source	Dest	Protocol	Pkts	Status
08:30	Wi-Fi Aziendale	Proxy / Gateway	HTTPS	150	ok
08:31	Proxy / Gateway	Portale Web	HTTPS	145	ok
08:35	Server Web	Active Directory	LDAP	12	ok
08:40	Server Web	Blockchain Tor Vergata	REST API	5	ok
09:00	Jump Server	Server SCADA	RDP	450	ok
09:15	Server SCADA	Tablet Aziendali	Proprietary	85	ok
10:45	Wi-Fi Aziendale	Proxy / Gateway	HTTPS	8500	ok
10:46	Proxy / Gateway	Portale Web	HTTPS	5	<i>retry</i>
10:47	Proxy / Gateway	Portale Web	HTTPS	8000	fail
11:00	Server Web	Active Directory	LDAP	5	fail
11:01	Server Web	Active Directory	LDAP	150	fail
11:15	Wi-Fi Aziendale	Blockchain Tor Vergata	REST API	25	fail
13:00	Proxy / Gateway	Jump Server	SSH	5	fail
13:05	Proxy / Gateway	Active Directory	LDAP	10	fail
14:00	Tablet Aziendali	Server Scada	HTTPS	45	ok
14:30	Server SCADA	Jump Server	TCP	60	ok
16:30	Server Web	Blockchain Tor Vergata	REST API	15	ok
16:45	Server Web	Active Directory	LDAP	8	ok
23:45	Server SCADA	Internet-Gateway	HTTP	20	fail
03:00	Jump Server	Server SCADA	SSH	500	ok

Tabella 19: Dataset di 20 eventi grezzi osservati nell'infrastruttura UniTorV E-Vote.

Step 4: Valutazione Preliminare

Procediamo misurando la distanza tra il comportamento osservato e la regione di comportamento atteso. In questa fase attribuiamo un livello di deviazione (lieve, media, forte) e ipotizziamo una causa radice, tenendo a mente che un'anomalia non coincide necessariamente con un attacco (potrebbe trattarsi di un guasto o di un'operazione di manutenzione).

Time	Source	Dest	Compatib.	Deviation	Causa
08:30	Wi-Fi Aziendale	Proxy / Gateway	Si	Null	Attività regolare
08:31	Proxy / Gateway	Portale Web	Si	Null	Attività regolare
08:35	Server Web	Active Directory	Si	Null	Attività regolare
08:40	Server Web	Blockchain Tor Vergata	Si	Null	Attività regolare
09:00	Jump Server	Server SCADA	Si	Null	Manutenzione tablet
09:15	Server SCADA	Tablet Aziendali	Si	Null	Manutenzione tablet
10:45	Wi-Fi Aziendale	Proxy / Gateway	Si	Alta	Attacco
10:46	Proxy / Gateway	Portale Web	Si	Bassa	Fault
10:47	Proxy / Gateway	Portale Web	Si	Alta	Attacco
11:00	Server Web	Active Directory	Si	Bassa	Unknown
11:01	Server Web	Active Directory	Si	Alta	Attacco
11:15	Wi-Fi Aziendale	Blockchain Tor Vergata	No	Alta	Attacco
13:00	Proxy / Gateway	Jump Server	Si	Media	Fault
13:05	Proxy / Gateway	Active Directory	No	Alta	Attacco
14:00	Tablet Aziendali	Server Scada	Si	Null	Attività regolare
14:30	Server SCADA	Jump Server	Si	Null	Attività regolare
16:30	Server Web	Blockchain Tor Vergata	Si	Null	Attività regolare
16:45	Server Web	Active Directory	Si	Null	Attività regolare
23:45	Server SCADA	Internet-Gateway	No	Alta	Attacco
03:00	Jump Server	Server SCADA	No	Alta	Attacco

Tabella 20: Valutazione preliminare degli eventi rispetto alla baseline.

Step 5: Classificazione Finale e Dato Utile

Poiché i sistemi di detection operano in condizioni di osservabilità parziale e sotto forte incertezza, una decisione rigorosa richiede contesto. Nella tabella seguente giustifichiamo la classificazione indicando quale *dato utile* risulterebbe dirimente per confermare o smentire l'ipotesi, specialmente per gli eventi etichettati come *Unknown*.

Evento (Time - Src → Dst)	Comp.	Score/Entropia	Class.	Dato utile
08:30 Wi-Fi Aziendale → Proxy / Gateway	Si	Basso	Normale	Nessuno, normale accesso
08:31 Proxy / Gateway → Portale Web	Si	Basso	Normale	Nessuno, normale accesso
08:35 Server Web → Active Directory	Si	Basso	Normale	Nessuno, normale accesso
08:40 Server Web → Blockchain Tor Vergata	Si	Basso	Normale	Nessuno, normale scrittura/lettura su blockchain
09:00 Jump Server → Server SCADA	Si	Basso	Normale	Nessuno, manutenzione in corso
09:15 Server SCADA → Tablet Aziendali	Si	Basso	Normale	Nessuno, normale invio dati al field
10:45 Wi-Fi Aziendale → Proxy / Gateway	Si	Alta	Attacco	Identificazione dell'IP/-MAC sorgente, analisi del volume di traffico (DoS)
10:46 Proxy / Gateway → Portale Web	Si	Bassa	Fault	Codici di errore HTTP (es. 503/504), colli di bottiglia e latenza
10:47 Proxy / Gateway → Portale Web	Si	Alta	Attacco	Analisi degli User-Agent
11:00 Server Web → Active Directory	Si	Bassa	Unknown	Username inserito, codice di errore specifico

Tabella 21: Valutazione delle deviazioni e classificazione finale (Parte 1 di 2).

Evento (Time - Src → Dst)	Comp.	Score/Entropia	Class.	Dato utile
11:01 Server Web → Active Directory	Si	Alta	Attacco	Lista degli account bersaglio, rate di tentativi falliti (Brute Force)
11:15 Wi-Fi Aziendale → Blockchain Tor Vergata	No	Alta	Attacco	IP sorgente bypass, endpoint API tentato, eventuali payload malevoli
13:00 Proxy / Gateway → Jump Server	Si	Media	Fault	Log degli errori del servizio SSH, utenza utilizzata
13:05 Proxy / Gateway → Active Directory	No	Alta	Attacco	Query LDAP inviata, payload per tentativo di privilege escalation
14:00 Tablet Aziendali → Server Scada	Si	Null	Attività regolare	Nessuno, normale comunicazione operativa
14:30 Server SCADA → Jump Server	Si	Null	Attività regolare	Nessuno, normale invio di status di sistema
16:30 Server Web → Blockchain Tor Vergata	Si	Null	Attività regolare	Nessuno, normale transazione (es. registrazione voto/-verbale)
16:45 Server Web → Active Directory	Si	Null	Attività regolare	Nessuno, normale verifica credenziali JWT/Login
23:45 Server SCADA → Internet-Gateway	No	Alta	Attacco	IP/URL di destinazione esterna, volume dati esfiltrati (potenziale C2)
03:00 Jump Server → Server SCADA	No	Alta	Attacco	Credenziali amministrative compromesse, comandi eseguiti fuori orario

Tabella 22: Valutazione delle deviazioni e classificazione finale (Parte 2 di 2).

API Monitoring

Baseline Path — CEO & MEMBRO + Albero Decisionale

Data: 3 giugno 2026 | CEO: luca.bianchi@example.com (wallet: 1) | MEMBRO: giulia.rossi@example.com (wallet: 2)

- Teal Successo / azione OK
- Purple Decisione / controllo
- Amber Audit / lettura
- Red Errore
- Blue Votazione

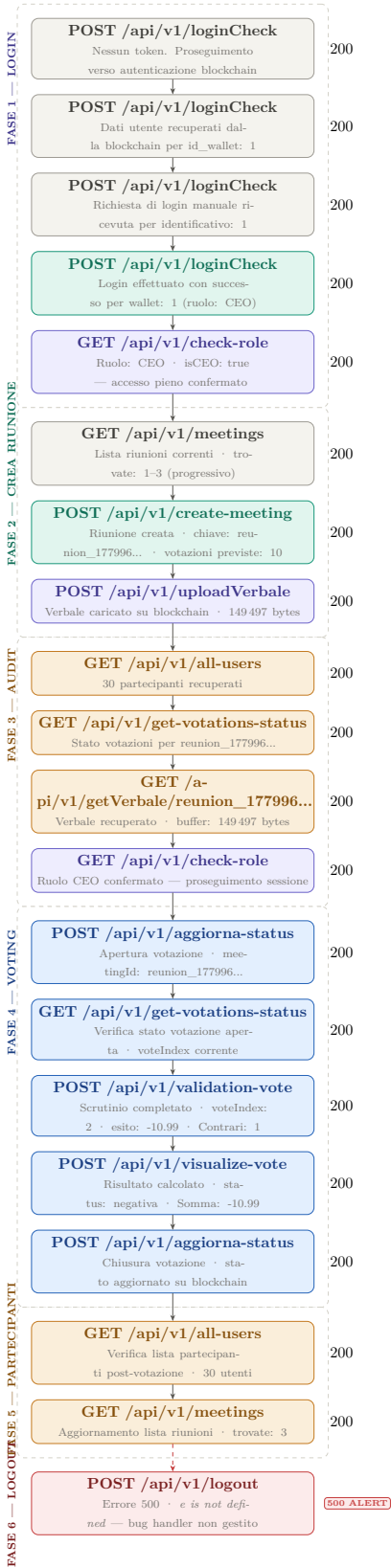
Endpoint rilevati nel log (28/05/2026)

Metodo	Endpoint	Ruolo	Descrizione
POST	/api/v1/loginCheck	CEO + MBR	Auth blockchain, emissione JWT, check password
GET	/api/v1/meetings	CEO + MBR	Lista riunioni per utente
GET	/api/v1/check-role	CEO + MBR	Verifica ruolo wallet (isCEO)
POST	/api/v1/create-meeting	CEO	Crea nuova riunione con N votazioni previste
POST	/api/v1/uploadVerbale	CEO	Carica verbale su blockchain
GET	/api/v1/getVerbale/{id}	CEO + MBR	Recupera verbale della riunione
GET	/api/v1/get-votations-status	CEO + MBR	Stato corrente delle votazioni
GET	/api/v1/all-users	CEO	Lista completa dei partecipanti (30 utenti)
POST	/api/v1/add-vote	MBR	Aggiunge un voto (favorevole/contrario/astenu-to)
POST	/api/v1/aggiorna-status	CEO	Apre o chiude una votazione
POST	/api/v1/validation-vote	CEO	Valida lo scrutinio di una votazione
POST	/api/v1/visualize-vote	CEO	Calcola e visualizza risultato finale
POST	/api/v1/logout	CEO + MBR	Logout effettuato volontariamente dall'utente.

Anomalie rilevate: **403** POST /add-vote — voto duplicato (hash già registrato) **404** GET /visualize-vote — votazione non ancora chiusa **401** POST /loginCheck — password errata

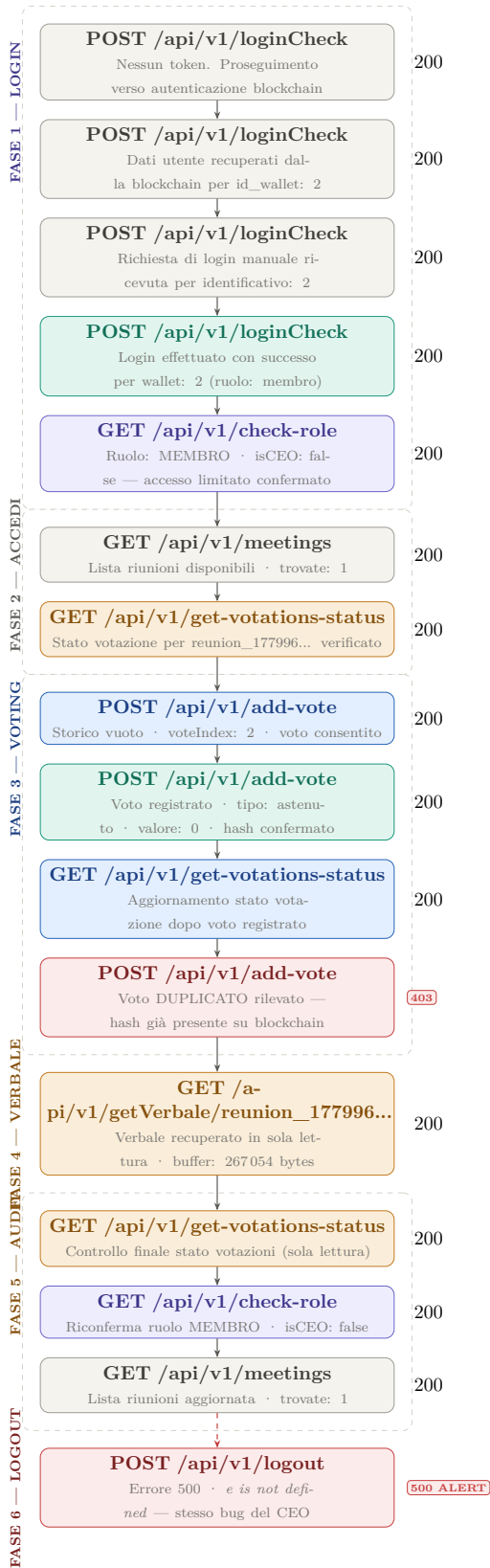
Baseline path — CEO

Utente: luca.bianchi@example.com · wallet: 1 · ruolo: CEO · isCEO: true

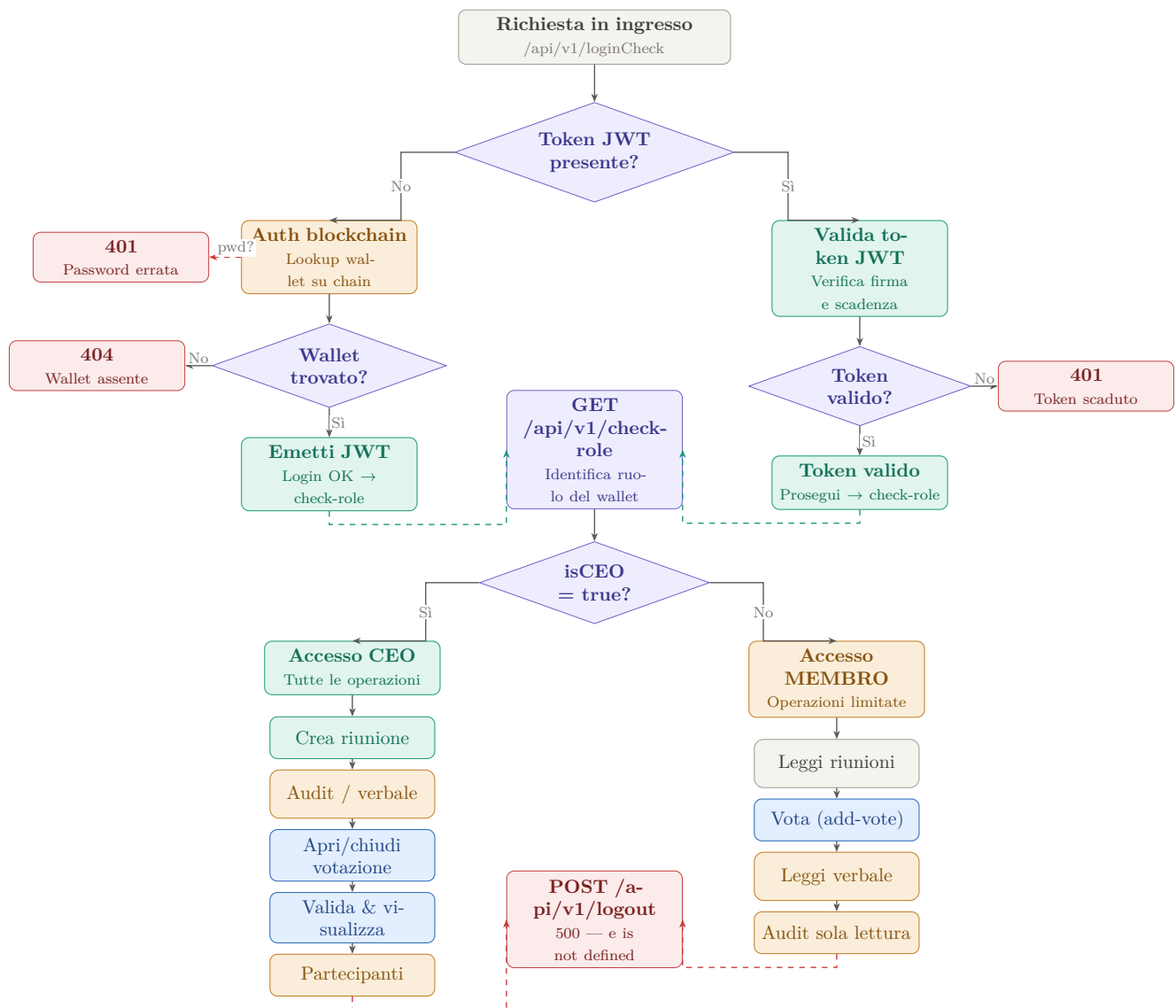


Baseline path — MEMBRO

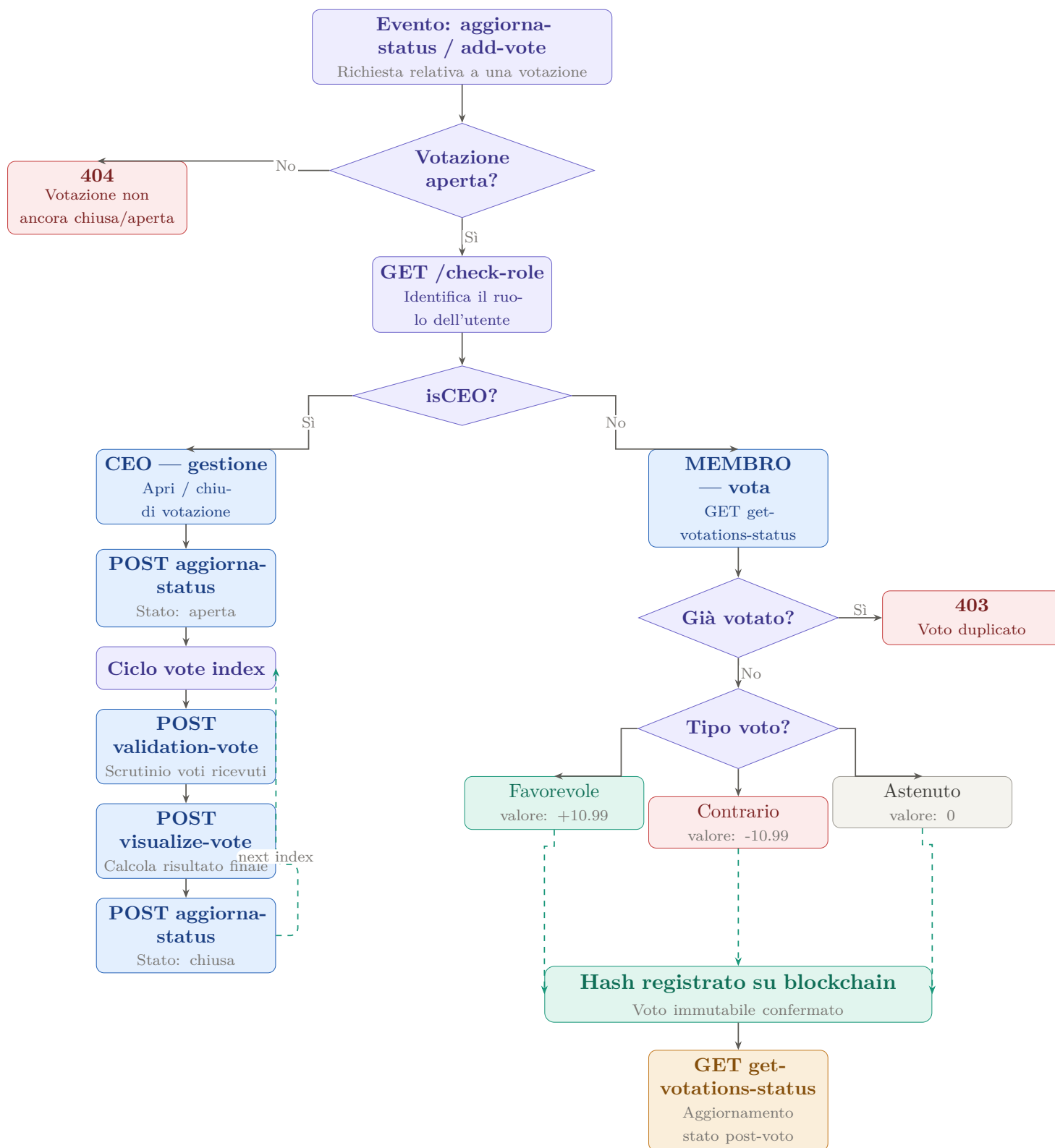
Utente: giulia.rossi@example.com · wallet: 2 · ruolo: MEMBRO · isCEO: false



Albero decisionale — Autenticazione e routing per ruolo



Albero decisionale — Ciclo di votazione



Riepilogo anomalie e raccomandazioni

Errori critici

- **[403] POST /api/v1/add-vote — Voto duplicato**
Il sistema rileva correttamente l'hash già presente su blockchain e rifiuta il secondo tentativo. Il comportamento è atteso ma andrebbe comunicato meglio lato UI.
Fix: mostrare un messaggio chiaro all'utente prima del secondo invio.
- **[401] POST /api/v1/loginCheck — Password errata**
Tentativo di login con credenziali errate per wallet: 1 e wallet: 2. Il sistema risponde correttamente con 401.
Raccomandazione: implementare rate-limiting dopo N tentativi falliti.
- **[404] GET /api/v1/visualize-vote — Votazione non ancora chiusa**
Il CEO ha tentato di visualizzare i risultati mentre la votazione era ancora aperta.
Fix: disabilitare il pulsante visualize nel front-end finché lo stato non è "chiusa".

Osservazioni sul flusso

- Il login avviene in **4 step sequenziali** per entrambi i ruoli (no-token → blockchain-lookup → richiesta-manuale → JWT).
- Il CEO ha eseguito **più sessioni** in sequenza nella stessa giornata (13:36, 13:40, 13:43, 13:44, 13:47) creando riunioni distinte con chiavi diverse.
- Il ciclo di voting del CEO segue il pattern ripetuto: `check-role` → `get-votations-status` → `aggiorna-status` → `validation-vote` → `visualize-vote` per ogni `voteIndex`.
- Il MEMBRO opera esclusivamente in **sola lettura** tranne che per `add-vote`; non ha accesso a `create-meeting`, `all-users`, `aggiorna-status`, `validation-vote`, `visualize-vote`.
- Il verbale del MEMBRO (267 054 bytes) è più grande di quello del CEO (149 497 bytes), suggerendo che le due sessioni fanno riferimento a riunioni differenti.

Legenda colori

Colore	Significato	Colore	Significato
Teal	Successo / azione completata	Blue	Operazione di votazione
Purple	Decisione / controllo ruolo	Red	Errore / anomalia
Amber	Audit / lettura / osservazione	Gray	Step neutro / intermedio